1

JAVASCRIPT

INTRODUCTION

2

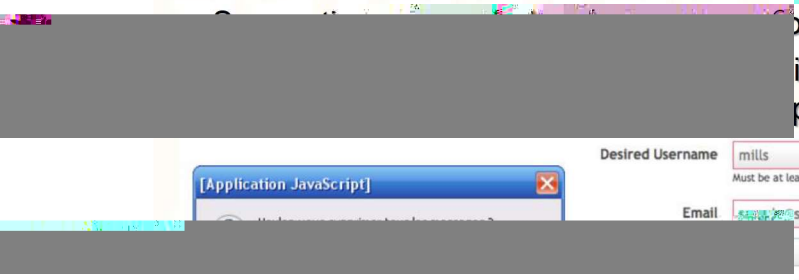
Javascript: c'est quoi ?

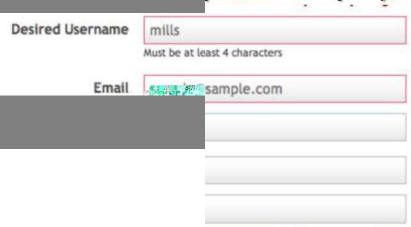
- Langage de script pour les pages web interactives
- Côté client : les scripts sont exécutés par le navigateur de l'utilisateur
- HTML pour le contenu, CSS pour la présentation, JavaScript pour programmer…
  - côté client seulement !
  - c'est-à-dire sur la page web récupérée par l'utilisateur (mais toujours même page récupérée)
- **Que peut-on faire en Javascript ?**
  - Puissance d'un langage de programmation complet, notamment structures conditionnelles (if/then/else), et boucles pour les répétitions (for, while).
  - Possibilité de changer le contenu de la page (modifier le HTML), les styles (modifier le CSS), d'afficher des boîtes de dialogue (messages d'alerte), de récupérer les événements clavier et souris (et réagir en conséquence).

Javascript: c'est quoi ?

Quelques exemples courants d'utilisation de JavaScript :

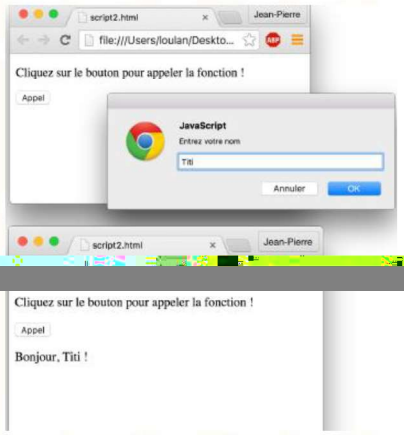
- Afficher des boîtes de dialogue pour confirmer une action
- Vérifier en temps réel (sans soumettre) que les informations sont valides lorsqu'on remplit les champs d'un formulaire (le mot de passe a le bon nombre de caractères, téléphone = que des chiffres…)
- [redacted]oteur de recherche comme le fait Google !
- [redacted]ique qui montre l'évolution de quelque [redacted]par exemple), etc.





[redacted]i ça ressemble ?

```
function maFonction()
{
let personne;
personne = prompt("Entrez votre nom", "Toto")
if (personne !== null && personne.trim() !== "")
       {
       document.getElementById("demo").innerHTML = "[redacted]
```




```
<p>Cliquez sur le bouton pour appeler la fonction !</p>
<button onclick="maFonction()">Appel</button>
<p id="demo"></p>
</body>
</html>
```

Insérer du javascript dans une page web

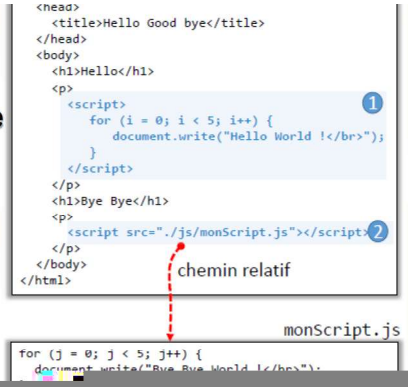
- Les script sont dans les balises **<script>...</script>**
- Avant HTML 5, il fallait utiliser <script type="text/javascript">...</script>. Le paramètre type n'est plus nécessaire maintenant, vous pouvez l'omettre.
- [redacted]ultiples, dans le header ou le body (sans importance)
- [redacted]éditeur de texte, comme pour du HTML
- Au lieu d'ajouter le code directement dans la page, possible de le mettre dans un fichier séparé (généralement avec l'extension .js) et de l'inclure avec: <script src="[redacted]></script>

Insérer du javascript

Deux possibilités pour un élément **<script>**

- soit il contient des instructions JavaScrip
- soit il pointe vers un fichier de script externe au travers

  un fichier texte qui contient le code JavaScript.
  - ➢ url vers un fichier du site : on indique le ./chemin
  - ➢ url vers un fichier d'un site externe: http:// ….
- fonctionnement équivalent à une intégration directe du code JavaScript à cet endroit.



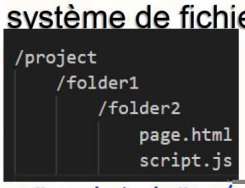
Quand l'attribut src est utilisé il ne faut rien écr
entre les balises <script src="…"> et </scr

Insérer du javascript dans une page web

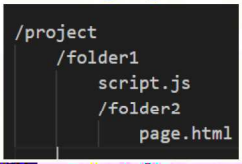
1. Chemin Absolu

Définition: C'est le chemin complet à partir de la racine du système de fichiers

C:\Users\Utilisateur\Documents\monscript.js




```
<script src="script.js"></script>
```

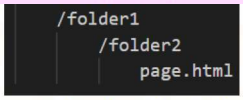


equel

le chemin est utilisé).

Exemple:

../monscript.js signifie : aller dans le répert

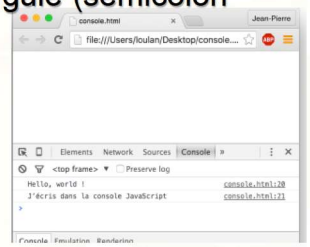



```
<script src="../../script.js"></script>
```

Javascript : éléments de base

- Toutes les instructions doivent être séparées par des points-virgule (semicolon en Anglais) :

```
console.log("Hello, world !");
console.log("J'écris dans la console JavaScript");
alert("Voici une alerte !");
document.write("bonjour tout le monde ");
```



- console.log() permet d'écrire dans la console JavaScript, pratique pour déboguer vos programmes (afficher la valeur d'une variable, ou voir si on est entré dans un if, par exemple)
- Pour afficher la console JavaScript :

Firefox : Menu Outils > Outils de navigateur > Développement de développement web > Console web

Sous Chrome, Edge et Firefox : F12

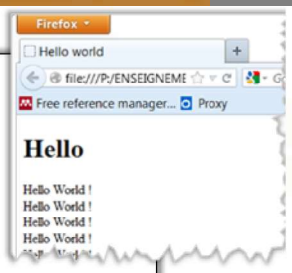
JavaScript dans un navigateur

- Code JavaScript peut être inclus

En quoi JavaScript est-il si différent d'au

HelloWorld.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello world</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>
      <script>
         for (i=0; i < 5; i++) {
            document.write("Hello World !</br>");
         }
      </script>
    </p>
  </body>
</html>
```

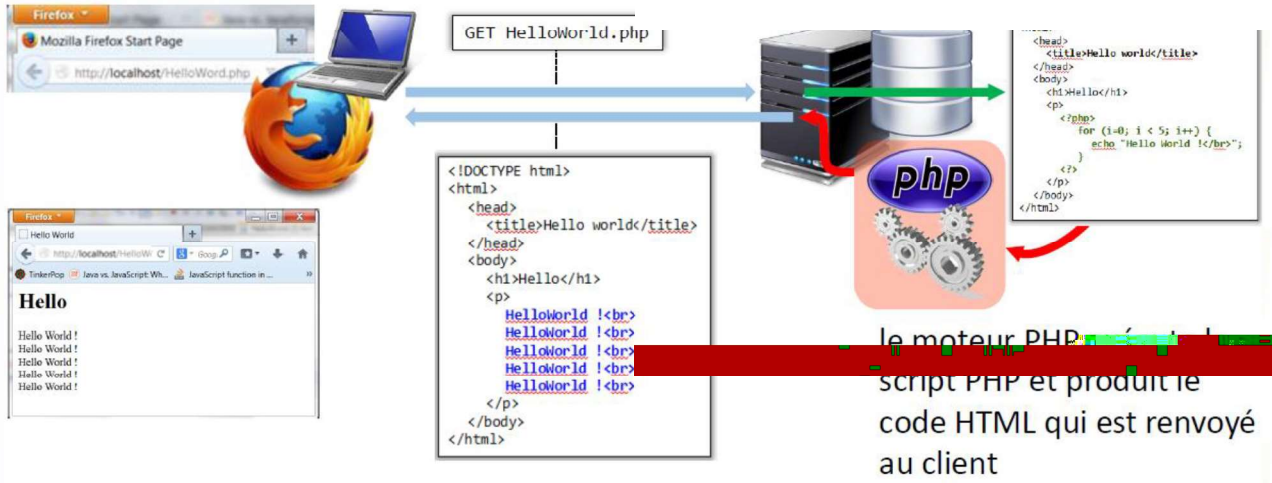


HelloWorld.php

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello world</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>
      <?php
         for ($i=0; $i < 5; $i++) {
           echo "Hello World !</br>";
         }
      ?>
    </p>
  </body>
</html>
```

JavaScript dans

- Un script PHP s'exécute côté serv



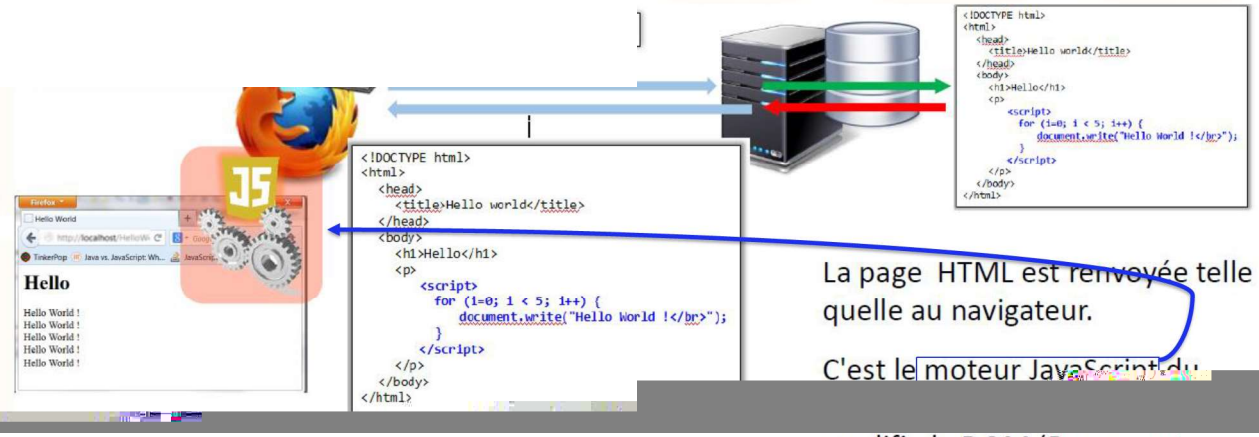
le moteur PHP
script PHP et produit le code HTML qui est renvoyé au client

Pr. Ghazouani Mohamed

11

JavaScript dans un navigateur

é client (dans le navigateur)



La page HTML est renvoyée telle quelle au navigateur.

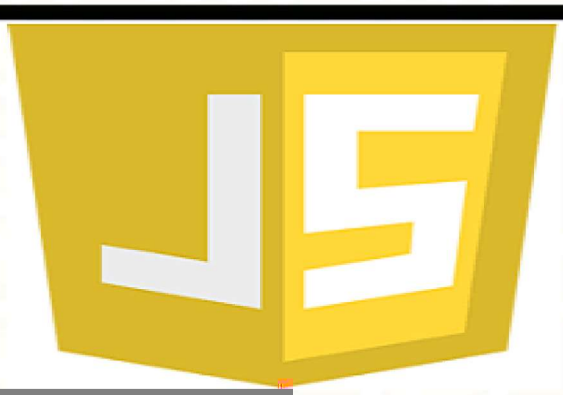
C'est le moteur JavaScript du

modifie le DOM (Document Object Model) de la page HTML

Pr. Ghazouani Mohamed

12

JAVASCRIPT



S DE BASE

Javascript : éléments de base

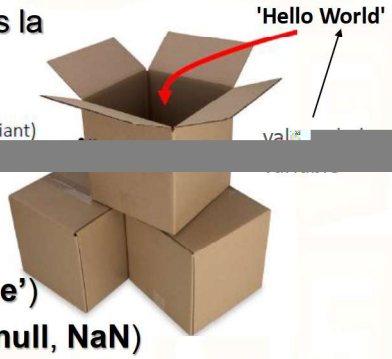
- **Commentaires** dans le code : soit entre /* et */ (lignes multiples possibles)
- Soit après // jusqu'à un retour à la ligne
- Rappel : tout ce qui est entre commentaire ignoré lors de l'exécution du code. Utile pour documenter le code, ou pour en « enlever » temporairement une partie quand vous travaillez dessus
- Même chose qu'en PHP, C++ ou Java par exemple…
- Exemple :

```
/* Je peux revenir
à la ligne facilement
comme ça pour les longs commentaires */
console.log("Hello World!"); // Commentaire court
```

Javascript : éléments de base

Variables

- Utilisées pour stocker temporairement des données dans la mémoire de l'ordinateur
- En JavaScript deux types de base :
  - primitifs
    - notation littérales
      - de nombres (**14**, **3.14159**, **3.25e-6**)
      - de chaînes de caractères (**'Ceci est une chaîne'**)
    - valeurs natives (built-in) (**true**, **false**, **undefined**, **null**, **NaN**)
  - objets
    - c'est



Javascript : éléments de base

Variables : règles de nommage

- une variable doit être désignée avec un nom unique (dans sa portée) : identifiant
- règles similaires à celles de la plupart des langages :
  - noms ne peuvent contenir d'espace, d'opérateurs arit
    caractère de ponctuation (.,;)
  - noms peuvent contenir des lettres, chiffres, _ ou $
  - noms ne peuvent commencer par un chiffre
  - noms sensibles à la casse (case sensitive) : maVariable vs mavariable
  - noms doivent être différents des mots réservés (par ex. var, if, while, function…).

Javascript : éléments de base

Variables : règles de nommage (Suite)

Bonnes pratiques

- choisir des noms descriptifs
  - par exemple pour une variable qui correspondant à une moyenne on préfèrera moyenne à m
- ne pas hésiter à utiliser plusieurs mots pour décrire avec précision le rôle d'un variable
  - par exemple pour une pour une variable qui correspondant à un prix unitaire d'un produit on préfèrera `prixUnitaire` à `pu`
- adopter des règles de nommage cohérentes dans tout le code

Définir un glossaire, réutiliser les mêmes termes partout. Conserver les mêmes conventions d'é

Pr. Ghazouani Mohamed

17

Javascript : éléments de base

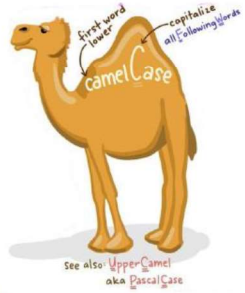
Variables : règles de nommage (Suite)

- Comment regrouper plusieurs mots dans un identifiant

Ex : variable représentant l'âge du capitaine. ~~ac~~ ~~agec~~ ~~agecap~~ ~~ageducapitaine~~

- Conventions d'écriture couramment utilisées

Camel case	Pascal case	Snake case	Kebab case
`ageDuCapitaine`	`AgeDuCapitaine`	`age_du_capitaine` `AGE_DU_CAPITAINE`	`age-du-capitaine`



Identifiants de classes de styles CSS
URL

18

Javascript : éléments de base

Déclaration multiple

var :

Vous pouvez déclarer plusieurs fois une variable avec le même nom dans la même portée, ce qui peut conduire à des comportements inattendus.

let :

Vous ne pouvez pas déclarer une variable avec le même nom dans la même portée.

```
var x = 10;
var x = 20;
console.log(x);

let x = 10;
let x = 20;
console.log(x);
```


Javascript : éléments de base

Déclaration des variables

- En JavaScript, **var** et **let** sont deux mots-clés utilisés pour déclarer des variables
  - **let** permet de déclarer des variables dont la portée est limitée à celle du bloc dans lequel elles sont déclarées.
  - Le mot-clé **var**, quant à lui, permet de définir une variable globale ou locale à une fonction (sans distinction des blocs utilisés dans la fonction).

```
function exempleVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // x est visible ici,
même si déclaré dans un bloc
}

exempleVar(); // Affiche 10
```

```
function exempleLet() {
  if (true) {
    let y = 20;
  }
  console.log(y); // Erreur : y n'est pas
défini ici
}

exempleLet(); // Cette ligne génère une
erreur
```

Javascript : éléments de base

Déclaration des variables

- JavaScript langage avec typage dynamique
  - ent de
  - ariable à l'exécution
- pas obligation d'initialiser une variable à sa déclaration
  - une variable non initialisée a la valeur **undefined**
- possibilité d'initialiser une variable à sa déclaration à l'aide d'expressions littérales

declaration.html

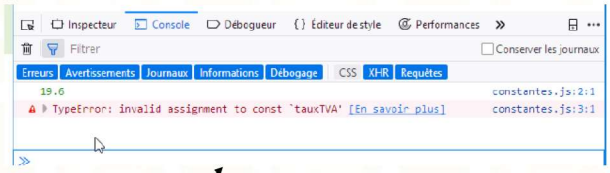
```
//Déclaration de a, de b et de c.
let a;
console.log(a);
let b,c;
//Affectation de a avec un entier.
a = 1;
console.log(a);
// Affectation de a avec une chaîne
a = "une chaîne de caractères";
console.log(a);
//forme littérale pour un entier
let quantite = 150;
console.log(prix);
//forme littérale pour un réel.
let tva= 19.6;
console.log(tva);
//forme littérale pour un tableau d'entiers
let tab1 = [ 1, 2, 3, 4 ];
console.log(tab1);
//forme littérale pour un objet
let etud= {"nom":"Belleli"
};
d);
console.log(etu
```


Javascript : éléments de base

Constante

- possibilité de déclarer des constantes en JavaScript
- utiliser **const** au lieu de **let**

```
const tauxTVA= 19.6;
let tauxReduc= 0;
let nbProduitsAchetes= 0;
...
if (nbProduitsAchete>= 5) {
% 
tauxTVA= 0.25;
}
...
```



une tentative de modification de la constante va provoquer une erreur d'exécution avec un message explicite sur la console

- Utiliser des majuscules pour les noms de constantes

```
Const TAUX_TVA = 0.2;
```

Javascript : éléments de base

Variables : Conversions numériques

- Relatifs aux valeurs et conversions des nombres, on trouve 4 fonctions :
  - isNaN détermine si un paramètre donné n'est pas un nombre
  - Number effectue une conversion
  - parseInt effectue une conversion en valeur entière
  - parseFloat effectue une conversion en valeur réelle
- Exemple
  - alert(isNaN(10));
  - alert(isNaN("10"));
  - alert(isNaN("blue"));
  - let num1 = Number("hello world");
  - let num2 = Number("00001");
  - let num3 = Number(true);
  - let num3 = parseInt("");
  - let num4 = parseInt(22.5);
  - let num6 = parseFloat("22.5");

Javascript : éléments de base

Variables : String

- On utilise les quotes simples (apostrophes) ou doubles (guillemets) ou ` (backticks) pour définir des valeurs chaînes de caractères. L'attribut length permet de déterminer la longueur d'une chaîne.
- Exemple
  - let nom = "Wilde";
  - let prenom = 'Oscar';
  - alert(prenom.length); // 6
  - let message = "toto a dit \"je suis malade\".";
  - let s1 = "Voilà l'automne";
  - let s2 = 'Il a dit : "Ah bon ! " ';
  - let nbHeures= 3;
- let s3 = ` durée en secondes : ${nbHeures* 60} s `; // "durée en secondes : 180 s"

${ … } permet d'inclure des variables et expressions dans une chaîne.
La valeur de l'expression est évaluée et son résultat devient partie de la chaîne.

Javascript : éléments de base

Variables : String

Il existe de nombreuses fonctions sur les chaînes de caractères.

- Exemple
  - let s = "hello world";
  - alert(s.length); // 11
  - alert(s.charAt(1)); // "e"
  - alert(s.slice(3)); // "lo world"
  - alert(s.slice(-3)); // "rld"
  - alert(s.substring(3,7)); // "lo w"
  - alert(s.indexOf("o")); // 4
  - alert(s.lastIndexOf("o")); // 7
  - alert(s.toUpperCase()); // HELLO WORLD
  - alert(s + " !"); // hello world !
- Il est possible de transtyper une valeur en chaîne avec String().
- Exemple
- let v1 = 10; alert(String(v1)); // "10"
- let v2 = true; alert(String(v2)); // "true"
- let v3 = null; alert(String(v3)); // "null"
- let v4; alert(String(v4)); // "undefined"

Javascript : éléments de base

Opérateurs

- JavaScript propose les opérateurs et instructions standards de C++/Java
  - Arithmétiques (+, -, *, /, %, ++, --, **)
  - Comparaisons (==, !=, <, >, <=, >=, ===, !==)
  - affectation (=,+=,-=,*=,/=,%=)
  - Logiques (&&, ||, !)

Exemple

```
let age = 10;
age++;
alert(age);
alert(age > 10 && age < 20);
alert(26 % 5);
age*=2;
alert(age);
```

```
let a = 5;
let b = "5";

// L'opérateur === vérifie si les valeurs sont strictement égales
if (a === b) {
  console.log("a est strictement égal à b");
} else {
  console.log("a n'est pas strictement égal à b");
}
```

Javascript : éléments de base

Conversion automatique des types

- JavaScript effectue des conversions automatiques de types
  - 50 * 20 ->1000
  - '50' * '20' -> 1000 // les valeurs de type string sont converties en valeurs de type number
  - '50' + '20' -> '5020' //l'opérateur +est surchargé quand les opérandes sont de type number c'est l'addition, quand ils sont de type string c'est la concaténation de chaînes
  - '50' + 20 -> '5020' //l'un des opérandes est de type string ->+ est la concaténation de chaînes, la valeur de type number est convertie en valeur type string
  - 50 + '20' -> '5020' //l'un des opérandes est de type string ->+ est la concaténation de chaînes, la valeur de type number est convertie en valeur type string
  - '50' -20 -> 30 //l'opérateur - est la soustraction, la valeur de type string est convertie en valeur type number
  - 50 + null -> 50 //la valeur null est convertie en la valeur 0 de type number
  - '50' + null -> '50null' //la valeur null est convertie en la valeur 'null' de type string
  - 50 + undefined -> NaN